

```

import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
np.random.seed(42)
n = 500
# Generate synthetic HR data with intentional problems
df = pd.DataFrame({
    'age': np.random.randint(22, 60, n).astype(float),
    'salary': np.random.randint(25000, 120000, n).astype(float),
    'dept': np.random.choice(['Eng', 'Sales', 'HR', 'Finance'], n),
    'education': np.random.choice(['HS', 'BSc', 'MSc', 'PhD'], n),
    'years_exp': np.random.randint(0, 35, n).astype(float),
    'left_job': np.random.randint(0, 2, n), # target: 1=left, 0=stayed
})
# Inject missing values (~12% each column)
for col in ['age', 'salary', 'dept']:
    df.loc[df.sample(frac=0.12).index, col] = np.nan
# Inject outliers
df.loc[0, 'salary'] = 999999 # extreme outlier
df.loc[1, 'age'] = 150 # impossible age
print('Shape:', df.shape)
print(df.isnull().sum())

```

```

Shape: (500, 6)
age      60
salary   60
dept     60
education 0
years_exp 0
left_job  0
dtype: int64

```

```

import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
np.random.seed(42)
n = 500
# Generate synthetic HR data with intentional problems
df = pd.DataFrame({
    'age': np.random.randint(22, 60, n).astype(float),
    'salary': np.random.randint(25000, 120000, n).astype(float),
    'dept': np.random.choice(['Eng', 'Sales', 'HR', 'Finance'], n),
    'education': np.random.choice(['HS', 'BSc', 'MSc', 'PhD'], n),
    'years_exp': np.random.randint(0, 35, n).astype(float),
    'left_job': np.random.randint(0, 2, n), # target: 1=left, 0=stayed
})
# Inject missing values (~12% each column)
for col in ['age', 'salary', 'dept']:
    df.loc[df.sample(frac=0.12).index, col] = np.nan
# Inject outliers
df.loc[0, 'salary'] = 999999 # extreme outlier
df.loc[1, 'age'] = 150 # impossible age
print('Shape:', df.shape)
print(df.isnull().sum())

```

```

Shape: (500, 6)
age      60
salary   60
dept     60
education 0
years_exp 0
left_job  0
dtype: int64

```

```

from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.linear_model import LogisticRegression

# 1. Define column groups
num_cols = ['age', 'salary', 'years_exp']
cat_cols = ['dept', 'education']

# 2. Fix impossible outliers

```

```

df['age'] = df['age'].clip(upper=100)
df['salary'] = df['salary'].clip(upper=200000)

# 3. Split BEFORE any fitting
X = df.drop('left_job', axis=1)
y = df['left_job']

X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 4. Numerical pipeline
num_pipe = Pipeline([
    ('impute', SimpleImputer(strategy='median')),
    ('scale', StandardScaler())
])

# 5. Categorical pipeline
cat_pipe = Pipeline([
    ('impute', SimpleImputer(strategy='most_frequent')),
    ('encode', OneHotEncoder(handle_unknown='ignore',
                              sparse_output=False))
])

# For older sklearn versions use:
# ('encode', OneHotEncoder(handle_unknown='ignore', sparse=False))

# 6. Combine preprocessing
preprocessor = ColumnTransformer([
    ('num', num_pipe, num_cols),
    ('cat', cat_pipe, cat_cols)
])

# 7. Full pipeline
model = Pipeline([
    ('prep', preprocessor),
    ('clf', LogisticRegression(max_iter=1000))
])

# 8. Train model
model.fit(X_tr, y_tr)

# 9. Evaluate
y_pred = model.predict(X_te)
clean_acc = accuracy_score(y_te, y_pred)

print(f"Clean pipeline accuracy: {clean_acc:.3f}")

```

Clean pipeline accuracy: 0.460

```

from sklearn.metrics import classification_report, confusion_matrix
y_pred = model.predict(X_te)
# Detailed metrics
print(classification_report(y_te, y_pred,
    target_names=['Stayed', 'Left']))
# Confusion matrix
print(confusion_matrix(y_te, y_pred))
# Cross-validate to get a more reliable estimate
from sklearn.model_selection import cross_val_score
scores = cross_val_score(model, X, y, cv=5, scoring='accuracy')
print(f'CV accuracy: {scores.mean():.3f} +/- {scores.std():.3f}')

```

	precision	recall	f1-score	support
Stayed	0.42	0.33	0.37	48
Left	0.48	0.58	0.53	52
accuracy			0.46	100
macro avg	0.45	0.46	0.45	100
weighted avg	0.45	0.46	0.45	100

[[16 32]
[22 30]]
CV accuracy: 0.432 +/- 0.070

